

Main is chapter ko **section-by-section Hinglish mein decode** karunga, lekin sirf translation nahi. Hum iska mental model samjhenge: **scope kya hai, JavaScript code run hone se pehle kya karta hai, target/source references kya hain, TDZ kyon aata hai aur lexical scope kaise variable lookup control karta hai.**

Chapter 1: “What’s the Scope?” ka detailed explanation

1. Scope ki zarurat kyon padti hai?

Variables banana simple lagta hai:

```
1 var name = "Ruth";  
2 let age = 25;
```

Lekin JavaScript engine ko kuch important decisions lene padte hain:

- name variable kahan available hai?
- Kya kisi function ke andar se name access ho sakta hai?
- Agar do alag jagah same naam ka variable ho, to kaunsa use hoga?
- Agar variable current function mein nahi hai, to engine use kahan search karega?
- Agar variable declare hi nahi hua, to kya hoga?

In sab questions ke rules ko collectively **scope** kaha jata hai.

Simple definition:

Scope rules decide karte hain ki program ke kisi particular location se kaunse variables accessible hain.

Example:

```
1 var name = "Global";  
2  
3 function showName() {  
4     var name = "Local";  
5     console.log(name);  
6 }  
7  
8 showName();  
9 console.log(name);
```

Output:

```
1 Local  
2 Global
```

Yahan same naam ke do variables hain:

- Global scope ka name
- showName() function scope ka name

Function ke andar local variable mil gaya, isliye engine ko global variable check karne ki zarurat nahi padi.

2. Is book ke three important concepts

Chapter ke introduction mein teen connected concepts diye gaye hain:

1. **Scope**
2. **Closure**
3. **Module pattern**

Scope

Scope decide karta hai ki variable ko **kahan se access** kiya ja sakta hai.

Closure

JavaScript functions first-class values hain. Iska matlab function ko:

- variable mein store kar sakte hain
- kisi function ko argument ke roop mein de sakte hain
- kisi function se return kar sakte hain

Example:

```
1  function outer() {  
2      var message = "Hello";  
3  
4      function inner() {  
5          console.log(message);  
6      }  
7  
8      return inner;  
9  }  
10  
11 var myFunction = outer();  
12 myFunction();
```

outer() execute hokar finish ho chuka hai. Phir bhi returned inner function message ko access kar sakta hai.

Ye **closure** hai.

Function apne original lexical scope ka connection maintain karta hai, chahe function ko baad mein program ke kisi aur location par execute kiya jaye.

Module pattern

Module pattern mein generally:

- kuch variables/functions private hote hain
- kuch methods public kiye jaate hain
- public methods closure ke through private data access karte hain

Example:

```
1  function createCounter() {
2      var count = 0;
3
4      return {
5          increment() {
6              count++;
7          },
8
9          getCount() {
10             return count;
11          }
12     };
13 }
14
15 var counter = createCounter();
16
17 counter.increment();
18 console.log(counter.getCount());
```

Yahan:

- count directly accessible nahi hai
- increment() aur getCount() public methods hain
- dono methods closure ke through private count access kar rahe hain

Chapter 1 ka main focus closure ya modules ko fully samjhana nahi hai. Abhi foundation banayi ja rahi hai: **scope ka structure kaise decide hota hai?**

3. Kya JavaScript interpreted language hai?

Generally beginners ko bataya jata hai:

JavaScript interpreted language hai aur code ko top-to-bottom line-by-line execute karta hai.

Ye explanation incomplete hai.

Modern JavaScript engine compilation aur interpretation dono tarah ki techniques use kar sakta hai. Lekin scope ko samajhne ke liye important mental model ye hai:

- | |
|----------------------------------|
| 1 Phase 1: Parsing / Compilation |
| 2 Phase 2: Execution |

JavaScript code directly pehli line se run karna start nahi kar deta. Execution se pehle engine code ko process aur analyze karta hai.

Isi initial processing mein engine roughly identify karta hai:

- variable declarations kahan hain
- function declarations kahan hain
- kaunsa variable kis scope ka hai
- blocks aur functions ka scope structure kya hai
- code mein invalid syntax to nahi
- strict-mode se related early error to nahi

Isliye book JavaScript ko **compiled language ke zyada close** consider karti hai.

Important distinction:

Yahan JavaScript ko compiled kehne ka matlab ye nahi ki C/C++ ki tarah hamesha pehle executable file generate hoti hai. Main point ye hai ki JavaScript code execution se pehle parse aur analyze hota hai.

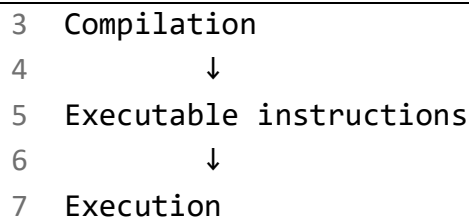
4. Compilation aur interpretation ka difference

Compilation

Traditional compilation mein generally poore source code ko pehle process karke machine-understandable instructions mein transform kiya jata hai.

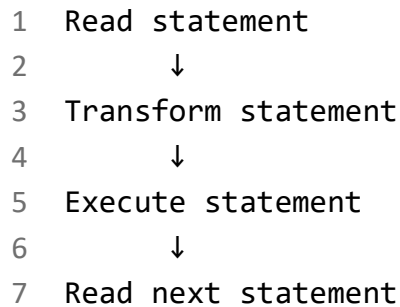
Conceptually:

- | |
|------------------------|
| 1 Complete source code |
| 2 ↓ |



Interpretation

Simple interpretation model mein:



Matlab ek statement transform aur execute hota hai, phir next statement par move kiya jata hai.

JavaScript ka actual case

Modern JavaScript engines kaafi complex hain. Woh use kar sakte hain:

- parsing
- bytecode generation
- interpretation
- Just-In-Time compilation
- runtime optimization
- hot code recompilation
- de-optimization

Lekin is chapter ke liye implementation details important nahi hain.

Aapko ye mental model yaad rakhna hai:

JavaScript execution se pehle code ko parse karta hai aur scope structure determine karta hai.

5. JavaScript code compile hone ke basic stages

Classic compiler theory ke according teen basic stages samjhaye gaye hain:

1. Tokenizing/Lexing

2. Parsing
3. Code Generation

Example:

```
1  var a = 2;
```

Stage 1: Tokenizing / Lexing

Engine code ke characters ko meaningful pieces mein divide karta hai. In pieces ko **tokens** bolte hain.

```
1  var a = 2;
```

Possible tokens:

```
1  var
2  a
3  =
4  2
5  ;
```

Har token language ke grammar mein koi meaning represent karta hai:

- var → keyword
- a → identifier
- = → assignment operator
- 2 → numeric value
- ; → statement terminator

Tokenizing aur lexing mein difference

Book ke according ye difference subtle aur academic hai:

- **Tokenizing** ko stateless identification ki tarah soch sakte hain
- **Lexing** mein current context ya state ka use ho sakta hai

Beginner ke liye in dono ke exact internal difference par zyada focus karna zaruri nahi.

Main point:

Source-code string ko meaningful language tokens mein break kiya jata hai.

Stage 2: Parsing

Parsing mein tokens ko language ke grammar ke according organize karke ek tree banaya jata hai.

Is tree ko kehte hain:

Abstract Syntax Tree, ya AST

Example:

```
1  var a = 2;
```

Iska simplified AST conceptually kuch aisa represent kar sakta hai:

```
1  VariableDeclaration
2  |— Identifier: a
3  |— NumericLiteral: 2
```

Actual AST format engine aur parser ke according zyada detailed ho sakta hai.

AST engine ko batata hai:

- ye variable declaration hai
- variable ka naam a hai
- isse numeric value 2 associate ki ja rahi hai

AST normal text nahi hai. Ye code ke **grammatical structure ka representation** hai.

For example, engine ko difference samajhna hota hai:

```
1  a = 2;
```

aur:

```
1  a == 2;
```

Characters similar hain, lekin grammatical meaning bilkul different hai.

Stage 3: Code generation

Ab AST ko executable instructions mein convert kiya jata hai.

For example:

```
1  var a = 2;
```

ke liye engine ko conceptually:

1. a naam ka identifier handle karna hai
2. uske liye required runtime storage arrange karni hai
3. value 2 ko a se associate karna hai

Actual engine bytecode ya machine instructions generate kar sakta hai. Exact implementation is chapter ka focus nahi hai.

6. JavaScript engine sirf teen steps nahi karta

Real JavaScript engines ka process in three textbook steps se kaafi complex hota hai.

Engine extra work bhi kar sakta hai:

- unnecessary operations ko optimize karna
- frequently executed code identify karna
- hot code ko dobara compile karna
- optimized machine instructions banana
- assumptions wrong hone par optimized code ko de-optimize karna

JavaScript compilation ke paas generally bahut kam samay hota hai, kyunki compilation aksar code run hone se just pehle hoti hai.

JIT kya hai?

JIT ka matlab hai:

Just-In-Time compilation

Engine code ko execution ke around compile karta hai. Frequently executed code ko baad mein aur aggressively optimize bhi kiya ja sakta hai.

Lekin scope samajhne ke liye JIT internals ya engine optimization learn karna required nahi.

7. JavaScript processing ke two required phases

Is chapter ka sabse important statement:

- | |
|----------------------------------|
| 1 Phase 1: Parsing / Compilation |
| 2 Phase 2: Execution |

Scope mostly first phase mein determine hota hai.

Is baat ke three observable proofs diye gaye hain:

1. Syntax errors
 2. Early errors
 3. Hoisting aur TDZ behavior
-

8. Proof 1: Syntax error execution se pehle milta hai

Example:

```
1 var greeting = "Hello";
2
3 console.log(greeting);
4
5 greeting = ".Hi";
```

Agar JavaScript genuinely pure line-by-line execute karta, expected behavior hota:

1. greeting mein "Hello" store hota
2. "Hello" print hota
3. third statement par syntax error milta

Lekin generally "Hello" print nahi hota. Program pehle hi SyntaxError throw kar deta hai.

Kyon?

Kyunki engine ne execution start karne se pehle poora relevant program parse kiya aur invalid syntax detect kar li:

```
1 greeting = ".Hi";
```

. yahan invalid position par hai.

Isse pata chalta hai:

Engine later line ki syntax ko execution start hone se pehle dekh chuka tha.

9. Proof 2: Early errors

Example:

```
1 console.log("Howdy");
2
3 saySomething("Hello", "Hi");
4
5 function saySomething(greeting, greeting) {
6     "use strict";
7     console.log(greeting);
8 }
```

Function mein duplicate parameter hai:

```
1 function saySomething(greeting, greeting)
```

Aur function strict mode mein hai:

```
1 "use strict";
```

Strict mode duplicate parameter names allow nahi karta.

Interesting cheez ye hai ki:

```
1 console.log("Howdy");
```

valid statement hone ke baad bhi "Howdy" print nahi hota. Engine execution shuru hone se pehle error throw kar deta hai.

Isse do important observations milte hain:

1. Engine ne parameter list ko analyze kiya
2. Engine ne function body mein "use strict" ko bhi recognize kiya

Engine ko ye dono pieces connect karne ke liye function ko properly parse karna pada.

Syntax error aur early error ka conceptual difference

Normal syntax error

Code language grammar ke hisaab se malformed hai:

```
1 greeting = ."Hi";
```

Early error

Tokens basic grammar ki tarah dikh sakte hain, lekin language ke contextual rules violate hote hain.

Example:

```
1 function saySomething(greeting, greeting) {  
2     "use strict";  
3 }
```

Strict mode rule duplicate parameter ko invalid banata hai.

Main takeaway:

Kuch errors execution ke dauran nahi, code parsing ke dauran hi detect ho jaate hain.

10. Proof 3: Hoisting aur Temporal Dead Zone

Example:

```
1 function saySomething() {  
2     var greeting = "Hello";  
3  
4     {  
5         greeting = "Howdy";
```

```
6      let greeting = "Hi";
7      console.log(greeting);
8    }
9  }
10
11  saySomething();
```

Aap initially soch sakte ho ki:

```
1  greeting = "Howdy";
```

outer function ke is variable ko update karega:

```
1  var greeting = "Hello";
```

Lekin aisa nahi hota. Error aata hai:

```
1  ReferenceError: Cannot access 'greeting' before initialization
```

Block ke andar kya ho raha hai?

Block:

```
1  {
2      greeting = "Howdy";
3      let greeting = "Hi";
4  }
```

let greeting poore block se associated hai. Isliye block mein pehli line ka greeting outer var greeting nahi hai.

Woh isi block ka let greeting hai:

```
1  greeting = "Howdy";
```

Lekin us waqt block-scoped greeting initialize nahi hua.

Isliye access TDZ mein hota hai.

Temporal Dead Zone kya hai?

Simplified definition:

Scope start hone aur let/const declaration ki initialization tak ke region ko TDZ kehte hain. Is region mein variable ko access karne par ReferenceError milta hai.

Conceptual view:

```
1  {
2      // greeting isi block ka variable hai,
3      // lekin abhi initialize nahi hua
4  }
```

```
5     greeting = "Howdy"; // TDZ error
6
7     let greeting = "Hi"; // initialization
8 }
```

Important correction

Common statement:

“let aur const hoist nahi hote.”

Book ke according ye accurate mental model nahi hai.

Agar let greeting hoisted ya scope se registered hi na hota, to pehli line outer var greeting ko find kar leti. Lekin engine pehli line par block wale greeting ko hi resolve karta hai.

Isse pata chalta hai:

- block-scoped binding already recognized hai
- lekin declaration execute hone se pehle initialized nahi hai
- isi wajah se TDZ error milta hai

Better statement:

let aur const declarations scope ke liye pehle se known hoti hain, lekin declaration ki initialization se pehle unhe access nahi kar sakte.

11. Kya parsing se compilation prove ho jati hai?

Strictly speaking, parsing aur compilation exactly same cheez nahi hain.

Theoretically engine ye kar sakta hai:

1. Source code ko AST mein parse kare
2. AST ko directly interpret kare
3. Separate compilation representation generate na kare

Ye technically possible hai.

Lekin book ka argument hai ki production-quality JavaScript engine ke liye AST ko efficient executable representation mein convert karna practical aur performance-friendly hai.

Isliye JavaScript processing:

Pure line-by-line textual interpretation ke comparison mein compilation model se zyada closely match karti hai.

Yahan terminology par debate karne se zyada important mental model hai:

```
1 Code pehle analyze hota hai
2     ↓
3 Scope structure determine hota hai
4     ↓
5 Uske baad execution hoti hai
```

12. Compiler Speak: Target aur Source references

Ab chapter batata hai ki variable ka har usage generally do roles mein se ek hota hai:

1. **Target**
2. **Source**

Purane compiler terminology mein inhe often kaha jata tha:

- LHS: Left-Hand Side
- RHS: Right-Hand Side

Lekin target/source better terms hain, kyunki assignment hamesha literal = ke left/right side mein visible nahi hota.

Target reference

Jis variable ko koi value assign ki ja rahi hai, woh target hai.

```
1 name = "Suzy";
```

Yahan:

```
1 name → target
```

Source reference

Jis variable se value retrieve/read ki ja rahi hai, woh source hai.

```
1 console.log(name);
```

Yahan:

```
1 name → source
```

Simple test:

Kya variable ko value di ja rahi hai? Haan, to target.
Kya variable se value li ja rahi hai? Haan, to source.

13. Students example ka scope analysis

Given program:

```
1  var students = [  
2    { id: 14, name: "Kyle" },  
3    { id: 73, name: "Suzy" },  
4    { id: 112, name: "Frank" },  
5    { id: 6, name: "Sarah" }  
6  ];  
7  
8  function getStudentName(studentID) {  
9    for (let student of students) {  
10      if (student.id == studentID) {  
11        return student.name;  
12      }  
13    }  
14  }  
15  
16  var nextStudent = getStudentName(73);  
17  
18  console.log(nextStudent);
```

Output:

```
1  Suzy
```

students declaration aur assignment

```
1  var students = [  
2    // ...  
3  ];
```

Is statement ko mental model ke liye do parts mein socha ja sakta hai:

Compilation phase

```
1  var students;
```

Engine students declaration ko scope ke saath register karta hai.

Execution phase

```
1  students = [/* objects */];
```

Array value students ko assign ki jati hai.

Assignment ke waqt:

```
1 students → target
```

Important: Ye conceptual separation hai. Iska matlab source code literally rewrite hona zaruri nahi.

Function declaration

```
1 function getStudentName(studentID) {  
2     // ...  
3 }
```

Yahan getStudentName special target-like association hai.

Compilation ke around:

- identifier getStudentName declare hota hai
- function value automatically us identifier ke saath associate hoti hai

Is behavior ko **function hoisting** kaha jata hai.

Isko roughly aise imagine kar sakte ho:

```
1 var getStudentName = function(studentID) {  
2     // ...  
3 };
```

Lekin book clearly kehti hai ki ye perfectly equivalent nahi hai.

Difference ye hai ki function declaration ka function association scope ke beginning mein available hota hai. Woh normal runtime = assignment ka wait nahi karta.

14. Saare target references

Chapter ke according example mein five target references conceptually identify kiye gaye hain.

Target 1: students

```
1 var students = [  
2     // ...  
3 ];
```

Array value students ko assign ho rahi hai.

```
1 students → target
```

Target 2: nextStudent

```
1 var nextStudent = getStudentName(73);
```

Function ka returned value nextStudent ko assign ho raha hai.

```
1 nextStudent → target
```

Target 3: student

```
1 for (let student of students) {
```

Har iteration mein students array ka next object student ko assign hota hai.

Conceptually:

```
1 student = nextValue;
```

Isliye:

```
1 student → target
```

Target 4: studentID

Function call:

```
1 getStudentName(73);
```

Yahan 73 argument function parameter studentID ko assign hota hai.

Conceptually:

```
1 studentID = 73;
```

Isliye parameter ek target hai:

```
1 studentID → target
```

Ye normal assignment ki tarah visible nahi hai, lekin function call ke waqt value parameter ko di ja rahi hai.

Target 5: getStudentName

```
1 function getStudentName(studentID) {
```

Function declaration ke identifier ko function value ke saath associate kiya jata hai.

```
1 getStudentName → special target association
```

15. Source references samajhna

Ab woh references dekho jahan value read ki ja rahi hai.

students as source

```
1 for (let student of students) {
```

Yahan:

- student → target, kyunki item usme assign ho raha hai
- students → source, kyunki array ki value read ho rahi hai

student and studentID as sources

```
1 if (student.id == studentID) {
```

Yahan:

- student ki value read ho rahi hai
- studentID ki value read ho rahi hai

Dono source references hain.

```
1 student    → source
2 studentID → source
```

student as source in return

```
1 return student.name;
```

student ki value read karke uski name property retrieve ho rahi hai.

```
1 student → source
```

getStudentName as source

```
1 getStudentName(73);
```

Function ko call karne se pehle engine ko getStudentName variable se function reference retrieve karna hai.

```
1 getStudentName → source
```

console and nextStudent as sources

```
1 console.log(nextStudent);
```

Yahan:

- console se object value retrieve ho rahi hai
- nextStudent se stored value retrieve ho rahi hai

```
1 console    → source
2 nextStudent → source
```

16. id, name, aur log variable references kyon nahi hain?

Example:

```
1 student.id
2 student.name
3 console.log
```

Yahan:

- student variable reference hai
- console variable reference hai
- id, name, aur log property names hain

Difference:

```
1 student.id
```

Isme engine pehle student variable resolve karta hai. Uske baad received object ke andar id property access karta hai.

Similarly:

```
1 console.log
```

Pehle console variable resolve hota hai. Phir us object ki log property retrieve hoti hai.

Isliye:

```
1 student → variable reference
2 id      → property
3
4 console → variable reference
5 log     → property
```

Bracket notation se difference aur clear hota hai:

```
1 student["name"];
```

Yahan "name" clearly ek string/property key hai, variable lookup nahi.

Lekin:

```
1 var propertyName = "name";
2 student[propertyName];
```

Yahan:

- student → variable source
- propertyName → variable source
- resulting "name" → property key

17. Target aur source distinction useful kyon hai?

Abhi ye sirf classification jaisa lag sakta hai, lekin variable lookup fail hone par target aur source ka difference important ho sakta hai.

For example:

```
1 console.log(missingVariable);
```

Yahan missingVariable source reference hai. Engine uski value read karna chahta hai. Declaration na milne par error hota hai.

Non-strict legacy behavior mein undeclared assignment alag tarah behave kar sakta tha:

```
1 missingVariable = 10;
```

Strict mode mein ye bhi error hai.

Chapter 2 mein failed lookup ke behavior ko deeper level par discuss kiya jayega.

Abhi ke liye:

Target lookup value store karne ke purpose se hota hai, source lookup value retrieve karne ke purpose se.

18. Runtime par scope modify karne ke “cheats”

Normally scope compile time par determine hota hai. Lekin non-strict mode mein do historical features runtime par scope ke behavior ko modify kar sakte hain:

1. eval()
2. with

Book strongly recommend karti hai ki dono avoid karo.

eval() scope ko kaise affect karta hai?

Example:

```
1 function badIdea() {  
2     eval("var oops = 'Ugh!';");  
3     console.log(oops);  
4 }
```

```
5
6  badIdea();
```

Output:

```
1  Ugh!
```

eval() ko ek string mili:

```
1  "var oops = 'Ugh!';"
```

Ye string normal data ki tarah receive hui, lekin eval() isse JavaScript code ke roop mein compile aur execute karta hai.

Non-strict context mein is string ke andar ki declaration:

```
1  var oops = "Ugh!";
```

current function scope ko affect kar sakti hai.

Without eval():

```
1  function badIdea() {
2      console.log(oops);
3  }
4
5  badIdea();
```

oops declared nahi hai, so ReferenceError milega.

eval() problematic kyon hai?

1. Readability problem

Source code ko dekhkar clear nahi hota ki kaunse variables runtime par appear ho sakte hain.

2. Performance problem

Engine ne jo scope structure pehle analyze aur optimize kiya tha, dynamic code uske assumptions ko disturb kar sakta hai.

3. Maintainability problem

String ke andar hidden declarations debugging ko difficult banati hain.

4. Security risk

Agar untrusted input eval() tak pahunch jaye, arbitrary code execution ka danger ho sakta hai.

Chapter specifically declaration-creating eval() ko avoid karne par focus karta hai.

with kya karta hai?

Example:

```
1  var badIdea = {
2      Oops: "Ugh!"
3  };
4
5  with (badIdea) {
6      console.log(Oops);
7  }
```

with object ki properties ko temporary identifier-like lookup environment ki tarah expose karta hai.

Block ke andar:

```
1  Oops
```

ko effectively:

```
1  badIdea.Oops
```

ki tarah resolve kiya ja sakta hai.

Problem ye hai ki variable resolution ambiguous ho jati hai.

Example:

```
1  var count = 10;
2
3  var obj = {
4      count: 20
5  };
6
7  with (obj) {
8      console.log(count);
9  }
```

Block ko separately read karke immediately clear nahi hai ki count:

- local variable hai
- outer variable hai
- ya obj ki property hai

Isse code confusing aur optimization difficult ho jati hai.

Strict mode ka role

Strict mode with ko allow nahi karta. Direct eval() ke scope-modifying behavior par bhi restrictions hoti hain.

Practical rule:

- 1 eval() se declarations create mat karo
- 2 with ka use bilkul mat karo
- 3 strict mode ya modern modules prefer karo

Modern JavaScript modules automatically strict mode mein execute hote hain, lekin chapter ka focus strict-mode concept par hai.

19. Lexical scope kya hai?

Ab chapter ke main conclusion par aate hain.

Lexical scope ka matlab hai ki scope source code mein functions, blocks aur declarations ki physical placement se decide hota hai.

Example:

```
1 var outer = "Outside";
2
3 function example() {
4     var inner = "Inside";
5
6     console.log(outer);
7     console.log(inner);
8 }
```

example() ko outer access hai, kyunki function global scope ke andar lexically placed hai.

Scope relationship:

```
1 Global Scope
2   └─ outer
3   └─ example
4       └─ Function Scope
5           └─ inner
```

Function scope apne outer scope ko access kar sakta hai.

Lekin outer scope inner scope ke variables ko directly access nahi kar sakta:

```
1 function example() {
```

```
2     var inner = "Inside";
3 }
4
5 console.log(inner);
```

Result:

```
1 ReferenceError
```

Kyunki inner sirf example() ke function scope mein declared hai.

20. var vs let/const scope placement

var function-scoped hota hai

```
1 function test() {
2     {
3         var message = "Hello";
4     }
5
6     console.log(message);
7 }
8
9 test();
```

var message ordinary block tak limited nahi hai. Woh enclosing function scope se associated hai.

Conceptual structure:

```
1 test Function Scope
2 └─ message
```

Output:

```
1 Hello
```

let aur const block-scoped hain

```
1 function test() {
2     {
3         let message = "Hello";
4         console.log(message);
5     }
6
7     console.log(message);
```

```
8 }  
9  
10 test();
```

Second console.log() par error milega, kyunki message nearest block se associated hai.

Conceptual structure:

```
1 test Function Scope  
2   └ Inner Block Scope  
3     └ message
```

Practical distinction:

```
1 function demo() {  
2   if (true) {  
3     var a = 1;  
4     let b = 2;  
5     const c = 3;  
6   }  
7  
8   console.log(a); // 1  
9   console.log(b); // ReferenceError  
10  console.log(c); // ReferenceError  
11 }
```

21. Variable lookup kaise hota hai?

Jab engine kisi variable reference ko resolve karta hai, to search current scope se start hoti hai.

Agar current scope mein variable nahi mila:

1. Next outer scope check hota hai
2. Phir uska outer scope check hota hai
3. Search global scope tak continue hoti hai
4. Matching declaration na mile to lookup fail hota hai

Example:

```
1 var a = 10;  
2  
3 function outer() {  
4   var b = 20;  
5
```

```

6      function inner() {
7          var c = 30;
8
9          console.log(a);
10         console.log(b);
11         console.log(c);
12     }
13
14     inner();
15 }
16
17 outer();

```

inner() ke perspective se lookup:

c lookup

```

1  inner scope → found

```

b lookup

```

1  inner scope → not found
2  outer scope → found

```

a lookup

```

1  inner scope → not found
2  outer scope → not found
3  global scope → found

```

Scope chain:

```

1  Global Scope
2  └─ a
3  └─ outer
4      └─ Outer Function Scope
5          └─ b
6              └─ inner
7                  └─ Inner Function Scope
8                      └─ c

```

Lookup inward direction mein nahi hota

Outer function inner function ke variables access nahi kar sakta:

```

1  function outer() {
2      function inner() {
3          var secret = "hidden";

```

```
4     }  
5  
6     inner();  
7     console.log(secret);  
8 }
```

outer() ka scope lookup inner() ke private scope ke andar enter nahi karta.

Scope lookup generally:

```
1 Current scope → enclosing outer scope → global scope
```

Not:

```
1 Outer scope → arbitrary inner scope
```

22. Shadowing kya hoti hai?

Agar inner scope mein same naam ka variable ho, to woh outer variable ko shadow karta hai.

```
1 var message = "Global";  
2  
3 function showMessage() {  
4     var message = "Local";  
5     console.log(message);  
6 }  
7  
8 showMessage();
```

Output:

```
1 Local
```

Lookup:

```
1 Current function scope → message found
```

Variable current scope mein mil gaya, isliye engine outer message tak search continue nahi karta.

Isko **shadowing** kehte hain.

Isi principle ko TDZ example mein bhi dekha tha:

```
1 function saySomething() {  
2     var greeting = "Hello";  
3  
4     {
```

```
5      greeting = "Howdy";
6      let greeting = "Hi";
7  }
8 }
```

Block ka let greeting, outer var greeting ko poore block ke liye shadow karta hai. Pehli line par block binding initialize nahi hui, isliye outer variable select hone ke badle TDZ error milta hai.

23. Compilation mein scope map banta hai, actual scope kab banta hai?

Ye chapter ka subtle but important point hai.

Book ke according compilation phase:

- variable declarations identify karti hai
- scope boundaries identify karti hai
- har identifier ko appropriate scope ke saath associate karti hai
- lexical scope ka plan/map prepare karti hai

Lekin compilation ke time program execute nahi ho raha.

Isliye chapter distinction banata hai:

Compilation time

Scope ka **structure/map** identify hota hai.

Conceptually:

```
1 Global:
2   - students
3   - getStudentName
4   - nextStudent
5
6   getStudentName function:
7   - studentID
8
9   for block:
10  - student
```

Runtime

Jab relevant code execute hota hai, tab actual runtime lexical environments/scopes create hote hain.

Example:

```
1 function greet(name) {  
2     let message = "Hello " + name;  
3     return message;  
4 }  
5  
6 greet("Kyle");  
7 greet("Suzy");
```

Compilation mein greet ka scope structure ek baar identify ho sakta hai:

```
1 greet scope:  
2 - name  
3 - message
```

Lekin greet() har call par naya runtime environment banata hai.

First call:

```
1 name = "Kyle"  
2 message = "Hello Kyle"
```

Second call:

```
1 name = "Suzy"  
2 message = "Hello Suzy"
```

Dono calls same lexical blueprint follow karti hain, par runtime environments separate hote hain.

Best mental model:

Compilation architect ki blueprint banati hai. Runtime us blueprint ke according actual rooms create karta hai.

24. Complete process ko ek example se samjho

```
1 var topic = "JavaScript";  
2  
3 function display() {  
4     let message = "Learning " + topic;
```

```
5     console.log(message);  
6 }  
7  
8 display();
```

Phase 1: Parsing/compilation

Engine identify karta hai:

Global scope

```
1 topic  
2 display
```

display() function scope

```
1 message
```

Engine references ko roles bhi dekh sakta hai:

```
1 topic = "JavaScript";
```

- topic → target

```
1 message = "Learning " + topic;
```

- message → target
- topic → source

```
1 console.log(message);
```

- console → source
- message → source
- log → property

```
1 display();
```

- display → source

Phase 2: Execution

1. "JavaScript" value topic ko assign hoti hai
2. Function display() call hota hai
3. display ka runtime scope/environment create hota hai
4. topic current function scope mein nahi milta
5. Search outer/global scope mein jati hai
6. topic mil jata hai
7. "Learning JavaScript" message ko assign hota hai
8. Message print hota hai

Output:

25. Chapter ka complete mental model

Is chapter ko following flow ki tarah yaad rakho:

```
1  JavaScript source code
2      ↓
3  Tokenizing / Lexing
4      ↓
5  Parsing
6      ↓
7  AST
8      ↓
9  Scope structure identify hota hai
10     ↓
11  Declarations appropriate scopes se associate hoti hain
12     ↓
13  Code generation / engine processing
14     ↓
15  Execution
16     ↓
17  Runtime lexical environments create hote hain
18     ↓
19  Variables current se outer scope ki taraf resolve hote hain
```

26. Most important takeaways

1. Scope accessibility ke rules hain

Scope decide karta hai ki kisi location se kaunsa variable available hai.

2. JavaScript code execution se pehle parse hota hai

JavaScript ko sirf textual line-by-line interpreter samajhna inaccurate mental model hai.

3. Processing mein at least two conceptual phases hain

```
1  Parsing/Compilation → Execution
```

4. Scope structure compilation ke during determine hota hai

Functions, blocks aur declarations ki placement scope map banati hai.

5. Syntax errors execution se pehle detect ho sakti hain

Isliye invalid later statement, earlier valid statement ko execute hone se rok sakta hai.

6. Early errors contextual language rules hain

Strict mode mein duplicate parameters iska example hain.

7. let aur const ko simply “not hoisted” kehna misleading hai

Unki binding scope mein recognized hoti hai, lekin initialization se pehle TDZ mein hoti hai.

8. Variable references target ya source hoti hain

- | | |
|---|-------------------------------------|
| 1 | Target → value assign ho rahi hai |
| 2 | Source → value retrieve ho rahi hai |

9. Function arguments parameters ko assign hote hain

- | | |
|---|----------------------------------|
| 1 | <code>getStudentName(73);</code> |
|---|----------------------------------|

Isme 73 parameter studentID ko assign hota hai, so studentID target role mein hai.

10. Properties variables nahi hoti

- | | |
|---|---------------------------|
| 1 | <code>student.name</code> |
|---|---------------------------|

Yahan student variable reference hai aur name property hai.

11. eval() aur with avoid karne chahiye

Ye runtime scope behavior ko confusing banate hain aur optimization/readability ko hurt karte hain.

12. JavaScript lexical scope use karta hai

Scope source code mein declarations, functions aur blocks ki placement se decide hota hai.

13. Variable lookup outward hota hai

- | | |
|---|--|
| 1 | Current scope → enclosing scope → global scope |
|---|--|

14. Compilation scope ka map banati hai, execution environments runtime par bante hain

Ek function har call par naya runtime environment create kar sakta hai, lekin sab calls same lexical scope blueprint follow karti hain.

One-line chapter summary

JavaScript code run hone se pehle engine usse parse karke variables, functions aur blocks ka lexical scope map determine karta hai; runtime par target references mein values assign hoti hain aur source references ki values current scope se outer scope ki taraf lookup ki jati hain.

Explanation approach

Maine chapter ko literal sentence-by-sentence translate karne ke bajaye ek connected mental model mein arrange kiya hai. Important distinctions, especially **compilation vs execution, target vs source, hoisting vs TDZ, property vs variable, aur compile-time scope map vs runtime environment**, ko examples ke through clarify kiya hai.